Lambda Layers and Dependencies

Explore how to optimize AWS Lambda function dependencies by understanding the use of deployment packages, container images, and Lambda layers. Learn to package and share code effectively across functions, automate layer updates, and manage versioning to build scalable and maintainable serverless applications.

This lesson introduces the evolution of dependency management in AWS Lambda functions. We begin by exploring the traditional approach of bundling dependencies directly within the deployment package, identifying its limitations and challenges. We then transition to AWS Lambda layers, a powerful mechanism for extracting shared code, improving modularity, and optimizing serverless deployments.

The need for external libraries in Lambda

In real-world development, Lambda functions often require third-party libraries that are not included in the minimal AWS Lambda runtime. While AWS provides some built-in libraries like `boto3` for Python, many applications rely on packages such as `requests`, `numpy`, `lodash`, or `uuid` to function properly.

To accommodate these libraries, one approach is to bundle dependencies directly with the function code in a single ZIP deployment package. Alternatively, we can package the function code and dependencies as a container image. Both methods are deployment package approaches. The difference lies in how the code and its dependencies are structured and deployed. Let's walk throug ZIP-based approach first.

Ask

Packaging dependencies with our Lambda function using a ZIP file

When we need additional packages not available in the standard Lambda runtime, we install them locally and upload them in the function as a `.zip` file. This method is straightforward for small-scale deployments.

Here is what the directory structure typically looks like:

```
my-function/
├── lambda_function.py    # Code file
├── requests/             # Installed dependency
```

Lambda function deployment package structure

To implement this setup, we can follow these steps:

```
mkdir my-function && cd my-function
pip install requests -t .
zip -r function.zip .
```

Create the deployment package

The function can then be deployed using the AWS CLI:

```
aws lambda create-function \\
  --function-name myFunction \\
  --runtime python3.11 \\
  --handler lambda_function.lambda_handler \\
  --zip-file fileb://function.zip \\
  --role arn:aws:iam::<Account-ID>:role/lambda-execution-role
```

Create a Lambda function with a ZIP file deployment package

Ask

Packaging dependencies with our Lambda function using a container image

The container image approach is an alternative method to deploy AWS Lambda functions. Instead of zipping the source code and dependencies, we build a Docker image and push it to Amazon Elastic Container Registry (Amazon ECR). This method is particularly useful when the dependencies are large or need native binaries that are hard to package using the ZIP method.

To use container images, we must:

1. Create a Dockerfile using an AWS-provided base image.
2. Copy our application code and install any needed packages.
3. Push the built image to ECR.
4. Reference the image in a Lambda function configuration.

Here is a basic example of a Dockerfile for a Python Lambda function:

```
FROM public.ecr.aws/lambda/python:3.11

# Copy function code
COPY app.py ${LAMBDA_TASK_ROOT}

# Install dependencies
COPY requirements.txt .
RUN pip install -r requirements.txt --target "${LAMBDA_TASK_ROOT}"

# Set the handler
CMD ["app.lambda_handler"]
```

Sample Dockerfile

After building the image with `docker build -t my-lambda-image .`, we tag and push it to ECR:

```
docker tag my-lambda-image:latest <Account-ID>.dkr.ecr.us-east-1.amazona
docker push <Account-ID>.dkr.ecr.us-east-1.amazonaws.com/my-lambda-image
```

Deploy image to ECR repository

Then we create the Lambda function and use the container image for code and dependencies:

```
aws lambda create-function \
  --function-name myContainerFunction \
  --package-type Image \
  --code ImageUri=<Account-ID>.dkr.ecr.us-east-1.amazonaws.com/my-lambda
  --role arn:aws:iam::<Account-ID>:role/lambda-execution-role
```

Create a Lambda function with a container image as a deployment package

This method provides more flexibility and allows us to work with larger and more complex dependencies than the ZIP method.

The issues with the bundled dependencies approach

While packaging dependencies with the function works initially, it quickly becomes inefficient at scale. Several issues arise when multiple functions or teams rely on this pattern.

- Lambda deployment packages have strict limits: 50 MB compressed and 250 MB uncompressed. Libraries like `pandas` or `scikit-learn` can exceed these limits rapidly.
- When each function includes its own copy of the same dependencies, it results in redundant storage usage. This redundancy increases deployment times and bandwidth consumption.
- Every time a dependency is updated, each function must be repackaged. This adds overhead to build and deployment processes and complicates version rollbacks.

Ask

- Tracking library versions and applying patches across many functions increases complexity. Each function must be manually maintained and redeployed for changes to take effect.

Introducing AWS Lambda layers

To resolve the inefficiencies of bundled dependencies, AWS introduced Lambda layers. A Lambda layer is defined as a ZIP archive that contains shared code, such as libraries, custom runtimes, or binaries.

When we attach a layer to a function, AWS mounts its contents into the `/opt` directory in the execution environment. The function code remains in `/var/task` but can reference dependencies from `/opt`. A function can include up to five layers, which are merged at runtime.

Using Lambda layers provides several benefits:

- They help separate concerns by keeping dependencies out of the function code.
- They promote reuse across functions, reducing duplication.
- They make it easier to update shared dependencies.
- They help stay within Lambda's 50 MB zipped and 250 MB unzipped package size limits by offloading large libraries like `pandas` or `scikit-learn` to a separate layer. This keeps the function package lightweight while still providing access to those libraries at runtime.
- They reduce the size of function deployment packages, accelerating deployments.

Use cases for Lambda layers

Ask

Lambda layers are ideal in scenarios where multiple functions need access to the same resources. They also support portability and standardization across teams.

Common use cases include:

- Externalizing common libraries like `requests`, `pandas`, or `lodash`.
- Sharing business logic components, such as logging or validation utilities.
- Distributing custom runtimes or interpreters.
- Managing compiled third-party binaries used in data processing or machine learning.

> **Note:** To stay cost-effective and performant, keep layers lean and focused. Layers are extracted and mounted at runtime. If the function references a large layer, or imports all of it even when most of it is unused, startup latency can increase, impacting performance-sensitive workloads.

How to use layers

To use Lambda layers, we must follow a three-step process. We start by packaging our dependencies into a ZIP file, then publish it as a layer, and finally attach it to a function.

The first step is to package the content:

```
mkdir -p python/lib/python3.11/site-packages/
pip install requests -t python/lib/python3.11/site-packages/
zip -r requests-layer.zip python
```

Create the required files and folders

Next, we publish the layer using the AWS CLI:

```
aws lambda publish-layer-version \\
  --layer-name requests-layer \\
  --zip-file fileb://requests-layer.zip \\
  --compatible-runtimes python3.11
```

Ask

Publish the Lambda layer

Finally, we attach the layer to a function:

```
aws lambda update-function-configuration \\
  --function-name myFunction \\
  --layers arn:aws:lambda:us-east-1:<Account-ID>:layer:requests-layer:1
```

Add the Lambda layer

To use the layer inside the function code, we may need to modify the import path as shown below:

```
import sys
sys.path.append('/opt/python')

import requests

def lambda_handler(event, context):
    response = requests.get("https://api.github.com")
    return response.json()
```

Sample Lambda function code

Automating Lambda layer updates in development workflows

As applications grow, manually managing Lambda layers becomes inefficient and error-prone. Automating layer updates streamlines the process of packaging, versioning, and applying shared dependencies across functions.

In a typical development workflow, this automation includes three steps:

1. **Package and version the layer:** A build step compresses the depen folder into a ZIP archive and publishes it as a new version of the layer.

Ask

2. **Track the new version:** The resulting version identifier or ARN is stored for use in subsequent deployment steps.
3. **Update dependent functions:** All relevant Lambda functions are reconfigured to use the new layer version automatically.

This logic is usually encapsulated in a script. Below is a simplified example of such a script:

```
# Build the layer
zip -r layer.zip python/

# Publish the layer and capture its version ARN
LAYER_ARN=$(aws lambda publish-layer-version \
  --layer-name shared-utils \
  --zip-file fileb://layer.zip \
  --compatible-runtimes python3.11 \
  --query 'LayerVersionArn' --output text)

# Update functions with the new layer version
aws lambda update-function-configuration \
  --function-name myLambdaFunction \
  --layers "$LAYER_ARN"
```

Script to automate layer updates

This script can be triggered automatically whenever shared code is modified. For example, it can run:

- After merging changes into a specific branch.
- As part of a nightly build process.
- Before deploying application code.

Automating layer updates helps enforce consistency, reduces manual overhead, and ensures that all functions remain aligned with the latest shared logic.

Ask

Sharing Lambda layers

Lambda layers can be shared across development teams or even across AWS accounts, enabling consistent use of shared libraries or utilities in collaborative environments. To share the layer, we can apply a resource-based policy using `add-layer-version-permission`. We can specify allowed principals using the ARN of the requesting entities.

For example, the following command grants another AWS account permission to use a specific layer version:

```
aws lambda add-layer-version-permission \
  --layer-name shared-utils \
  --version-number 3 \
  --statement-id share-layer \
  --action lambda:GetLayerVersion \
  --principal <Account_ID>
```

Resource based policy for a Lambda layer

> **Note:** To make a layer public, replace the `--principal` value with `*`.

Shared layers can then be referenced by their full ARN in functions across accounts.

Layer versioning and runtime compatibility

Every time we publish a new version of a layer, AWS creates an immutable version. Lambda functions do not automatically adopt newer versions—we must explicitly reference the desired version in the function configuration.

When using Lambda layers, we should adopt a clear versioning strategy:

- Pin specific layer versions in production to ensure consistent behavio
- Use semantic versioning (like, v1.0.0, v1.1.0) for clarity.
- Tag and track versions in a version control system.

Ask

Each layer must specify one or more compatible runtimes. If a layer is built for Python 3.11, it will not function properly with a function that uses Python 3.9. It is important to align the runtime environments accordingly.

Common errors and troubleshooting

Developers may encounter issues when working with Lambda layers. The following table outlines common problems, their causes, and recommended solutions:

Error	Reason	Troubleshooting
`ModuleNotFoundError`	Missing module or path issue	Check the structure and append `/opt/python` to `sys.path`
Incompatible runtime error	Function uses unsupported runtime	Rebuild layer or change function runtime
Layer too large	Exceeds AWS size limits	Remove unused files, split into multiple layers, or use smaller packages

Conclusion

Managing code dependencies in AWS Lambda involves choosing how to include the libraries and packages our functions rely on. These can be bundled directly with the code, built into container images for greater flexibility, or separated into layers to promote reuse and reduce duplication. Each option has its place,

Ask